

Transformation visual controllers for interactive scene manipulation in virtual reality

Bachelor Thesis



Author: Pavlos Ziotas (Student ID: 4208)

Supervisor: Prof. Papagiannakis Georgios

Co-Supervisor: Dr. Kamarianakis Manos

Department of Computer Science

School of Sciences & Engineering

University of Crete

Heraklion, Greece

February 18, 2024

Abstract

Scene manipulation is an essential feature that is provided by a variety of applications, like game engines. The most commonly used tools used for scene manipulation are gizmos. These tools provide a user friendly way to change the position, orientation and size of virtual objects. Meanwhile, virtual reality has become a trend in both the game and scientific industry, with many applications releasing every year.

The goal of this project is to develop the functionality gizmos offer and make use of this functionality inside virtual reality. The scene that was used for the demo is consisted of a floor, a table on which there is a teapot and a skybox. When the controllers are activated, we can also see the controller's location, represented by a cube for each hand. For the gizmos implementation we used ray-casting and intersection techniques in order to detect the intersection points on the gizmos components and compute the required transformation parameters. Finally, by refactoring this implementation so that it can acquire input from the virtual reality controllers, ray-casting was done from the right controller rather than the screen. In the end, the user could change the object they want to transform using the left controller's grab button and interact with the gizmos using the right controller's grab button.

Declaration of Authorship

I, Pavlos Ziotas, declare that this thesis titled, "Transformation visual controllers for interactive scene manipulation in virtual reality", and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signature: Pavlos Ziotas

Date: February 18, 2024

Acknowledgements

I would like to express my gratitude to my supervising professor Kamarianakis Manos for his continuous support and essential direction throughout the course of this project.

I would also like to thank Geronikolakis Stratos, Lydakis Nikos, Zikas Paul and Protopsaltis Antonis for their instructions and helpful advice.

Contents

1	Introduction	1
1.1	State of The Art	1
1.2	Problem Definition	2
1.3	Overview of our approach	2
2	Graphics	3
2.1	Overview	3
2.2	Platforms/Game Engines	3
2.3	Python: Elements	4
2.4	Our Contribution	4
2.4.1	Textures	4
2.4.2	OpenXR	5
3	Gizmos	6
3.1	Intro	6
3.2	Our Approach	7
3.3	Technical details	7
4	ElementsXR	12
4.1	Intro	12
4.2	Our Approach	12
4.3	Technical details	12
4.3.1	Connecting the scene to the OpenXR runtime	12
4.3.2	Gizmos in ElementsXR	13
5	Conclusion	16
5.1	Summary	16
5.2	Limitations and future work	16
	References	17
A	Appendix	18
A.1	OpenXR Instructions	18
A.2	Gizmos	20
A.3	OpenXR Options	22
A.4	ElementsXR Sample code	22

List of Figures

1	A graphics tablet, used for Digital Art	1
2	Data Visualization with computer graphics	1
3	Unity Editor	3
4	Unreal Engine 5 Editor	4
5	A texture-less cube	4
6	A Cube painted with the UOC logo	5
7	A wooden cube with a background	5
8	Example of translation gizmos	6
9	Example of rotation gizmos	6
10	Example of scaling gizmos	7
11	Ray does not hit the bounding box.	8
12	Ray hits the bounding box.	9
13	Translation gizmos in Elements	10
14	Rotation gizmos in Elements	10
15	Scaling gizmos in Elements	11
16	Elements scene inside the OpenXR runtime.	13
17	Translation inside OpenXR.	14
18	Rotation inside OpenXR.	14
19	Scaling inside OpenXR.	15
20	OpenXR Example	19

List of Tables

1 Introduction

1.1 State of The Art

Computer Graphics is a sub-field of computer science that involves creating images using computers. It has revolutionized various industries, including video games, digital art, photography, 3D modeling, visual effects, virtual and augmented reality applications. Moreover, it has a significant impact on scientific applications like data visualization, physics simulations, and training. (see Fig 1 and 2).



Figure 1: A graphics tablet, used for Digital Art



Figure 2: Data Visualization with computer graphics

1.2 Problem Definition

Many computer graphics applications strive to provide an immersive user experience. One crucial aspect of achieving this goal is the environment that the user perceives while using these applications. Thus, manipulating the objects inside these environments can significantly assist in the development of such applications.

One way to address this issue is by using a graphical user interface (GUI) to manipulate virtual objects by modifying variables associated with the object's spatial information. However, this method requires the user to change each variable either by using their keyboard or by moving sliders, which may not be very interactive or comfortable.

To ensure a more interactive and engaging user experience, gizmos are considered the most popular methodology for scene manipulation in the software industry. Gizmos are tools that provide users with a more interactive approach to directly manipulate virtual scenes.

1.3 Overview of our approach

The objective of this thesis was to implement Gizmos' functionality and utilize it in Virtual Reality. Firstly, we developed the primary functionality on the screen. After completion, we connected the scene to the OpenXR runtime, enabling us to view the scene through virtual reality headsets. Lastly, we utilized Gizmos' properties in virtual reality by inheriting most of them while overriding some others.

2 Graphics

2.1 Overview

Computer Graphics is a sub-field of computer science that deals with manipulation of digital and geometric information. Some of its major sub-fields include geometry, animation and rendering. Thanks to the applications of those sub-fields over the past decades, a considerable amount of applications and tools was created. One example of such applications are game engines.

2.2 Platforms/Game Engines

Game engines (see Fig 3 and 4). are tools fundamentally designed for creating video games without building everything from the ground up. The main functionality that is provided by those tools may include a rendering engine for 2D and 3D graphics, a physics engine, sound framework, animation, Scripting, scene graph visualization, memory management and threading. Game Engines commonly provide a variety of visual development tools in addition to reusable software components. These tools are usually supplied with an integrated development environment that offers facilities that can be used for rapid development of games in a data-driven manner. The most commonly used programming languages used by Game Engines are c-Sharp and c++.

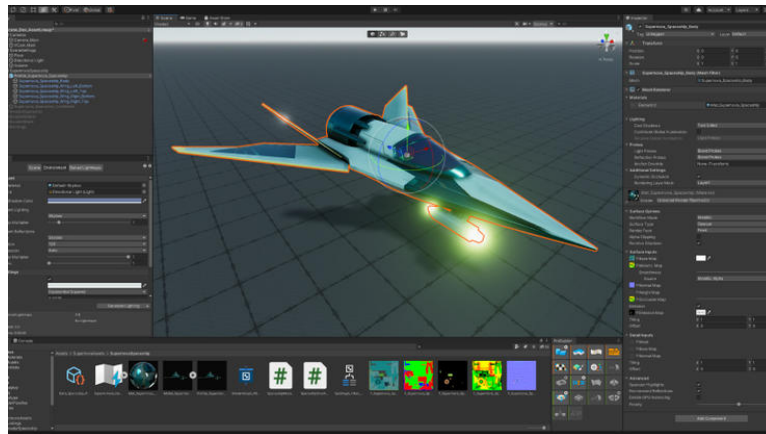


Figure 3: Unity Editor



Figure 4: Unreal Engine 5 Editor

2.3 Python: Elements

Elements is a tool that combines the principles of the entity-component system in with the features provided by the scene graphs. This tool is intended to provide the same features that modern game engines provide, and use them for topics like computer graphics and scientific visualization, or more advanced topics like geometric algebra.

2.4 Our Contribution

2.4.1 Textures

Initially Elements did not have support for textures. Therefore there were many limitations regarding what could be shown on a window (See Fig. 5).

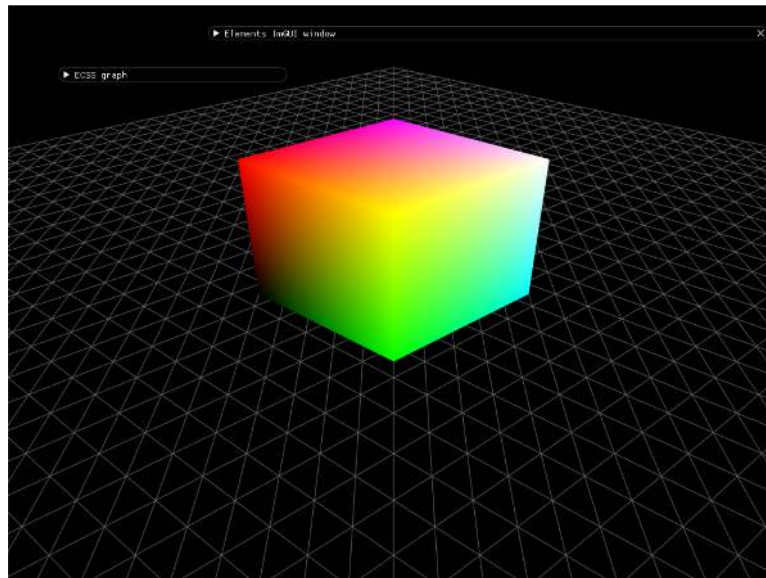


Figure 5: A texture-less cube

In order to achieve better visual results we added support for 2D and 3D texture mapping. The outcome was much better since the objects could be rendered using textures and not just colors (See Fig. 6 and 7).

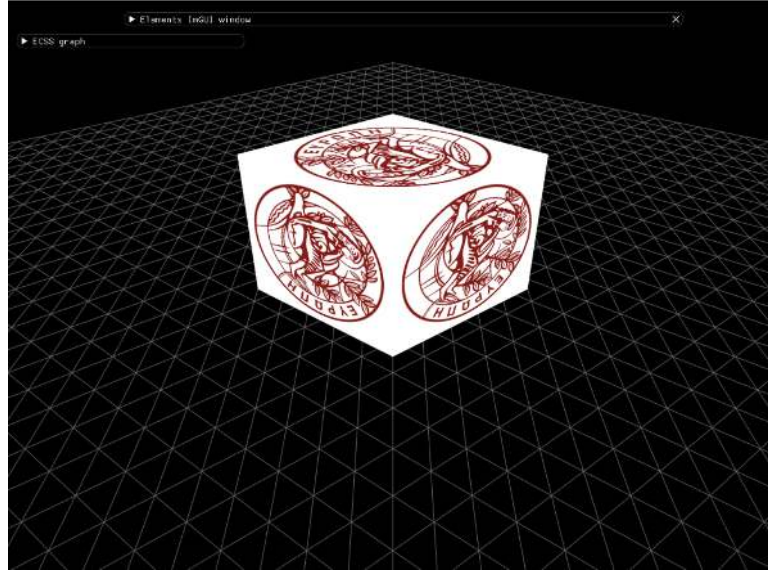


Figure 6: A Cube painted with the UOC logo

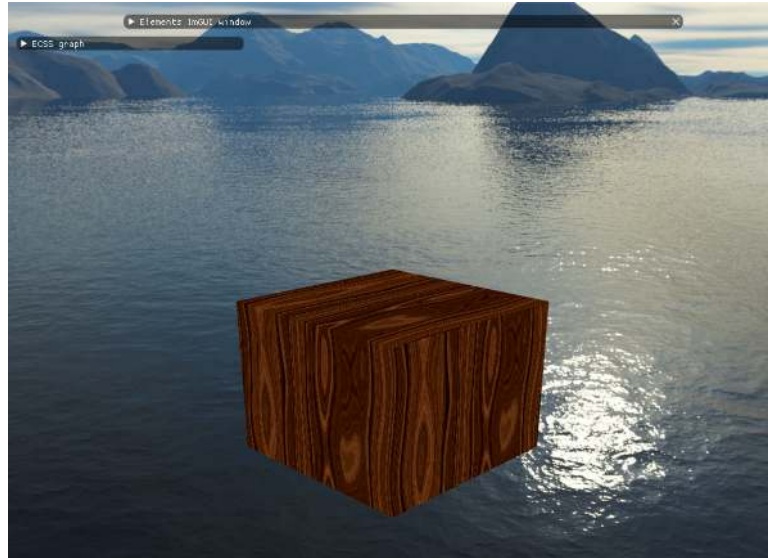


Figure 7: A wooden cube with a background

2.4.2 OpenXR

Furthermore, an investigation on how to build an OpenXR program from scratch was conducted. Instructions on the specific task are provided below (see Appendix A.1).

3 Gizmos

3.1 Intro

Gizmos are utilities used by game engines for visual debugging and scene manipulation. They provide a convenient way to apply transformations to entities on a scene. There are 3 types of gizmos in general.

- Translation gizmos allow us to move objects from one position to another (See Fig. 8).

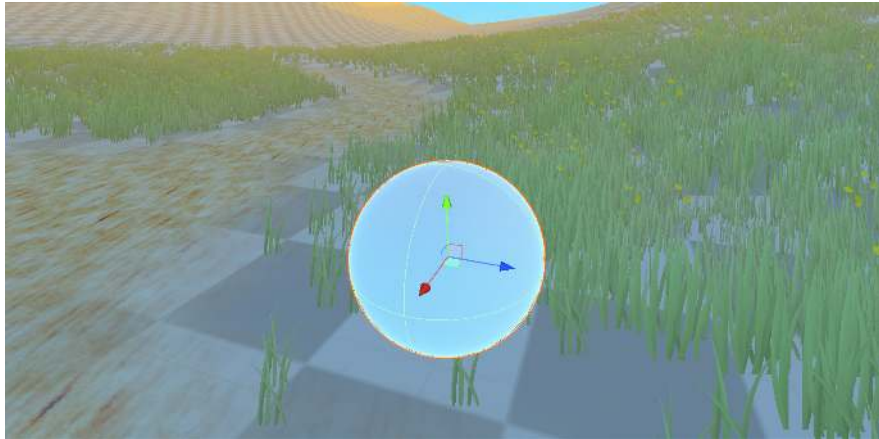


Figure 8: Example of translation gizmos

- Rotation gizmos allow us to rotate objects (See Fig. 9).

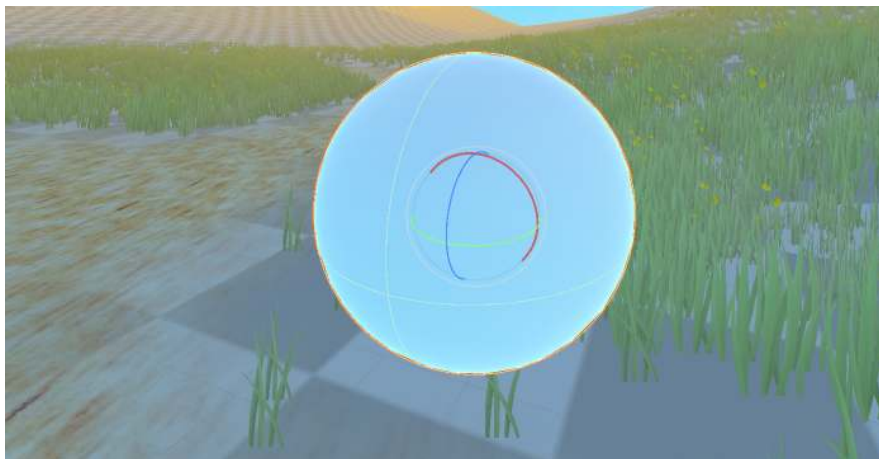


Figure 9: Example of rotation gizmos

- Scaling gizmos allows us to change the objects' size (See Fig. 10).

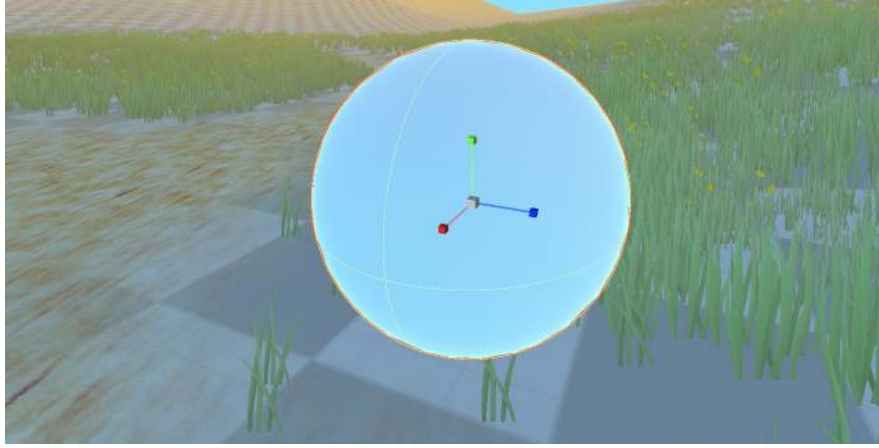


Figure 10: Example of scaling gizmos

3.2 Our Approach

The method we used for the gizmos in Elements is the ray-bounding-box intersection. Using this type of intersection, whenever the cursor hits a gizmo, we calculate and store the intersection point for each frame. So, we apply transformations based on the distance between subsequent intersection points, whenever the mouse cursor changes position.

3.3 Technical details

Firstly, we have to convert the cursor's screen coordinates to world coordinates. To accomplish this, we should store the main camera's projection and view matrices and calculate their inverse values. Since calculating an inverse matrix is an expensive operation, we only do so whenever the gizmos are enabled, and the previous matrices have not changed. After calculating the projection and view's inverse matrices we use the following steps to change the ray's screen coordinates to world coordinates and find its direction.

$$\begin{aligned}
 x &\leftarrow 2.0 * (mouse_x / windowWidth - 0.5) \\
 y &\leftarrow -2.0 * (mouse_y / windowHeight - 0.5) \\
 rayStart &\leftarrow (x, y, -1.0, 1.0) \\
 rayEnd &\leftarrow (x, y, 0.0, 1.0) \\
 rayStartWorld &\leftarrow InverseProjection * InverseView * rayStart \\
 rayEndWorld &\leftarrow InverseProjection * InverseView * rayEnd \\
 \mathbf{rayStartWorld} &\leftarrow \text{normalize}(rayStartWorld) \\
 rayEndWorld &\leftarrow \text{normalize}(rayEndWorld) \\
 rayDirection &\leftarrow rayEndWorld - rayStartWorld \\
 \mathbf{rayDirection} &\leftarrow \text{normalize}(rayDirection)
 \end{aligned}$$

Using The ray's starting point and direction in world space coordinates, we utilize a ray-bounding box intersection algorithm to determine whether a ray hits a gizmo's bounding box or not. The type of Bounding box we Use for this implementation, is the axis-aligned bounding box. This kind of bounding box makes use of a selected Object's minimum and maximum coordinates, so calculating one has $O(n)$ complexity, where n is the number of points an object is consisted of. When having to decide whether a ray hits a bounding box or not, there are 2 cases.

Let **tmin** and **tmax** be the minimum and maximum coordinates of any axis from a given axis-aligned bounding box respectively:

- if **tmin** is greater than **tmax**, there is no intersection (see Fig. 11).
- If **tmin** is smaller than **tmax** then the ray intersects with the bounding box (see Fig. 12).

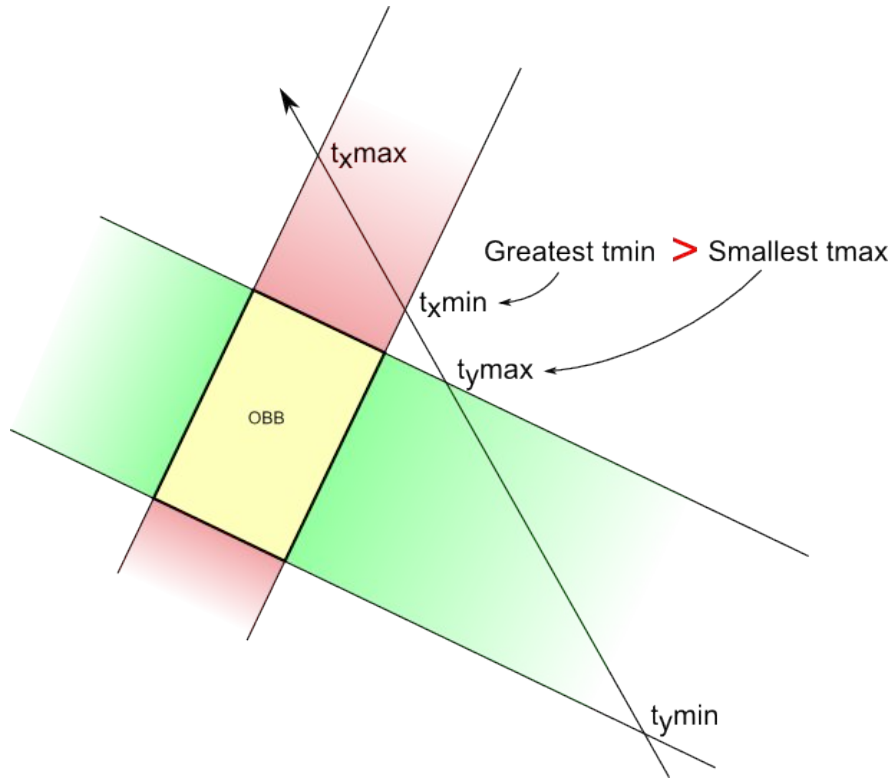


Figure 11: Ray does not hit the bounding box.

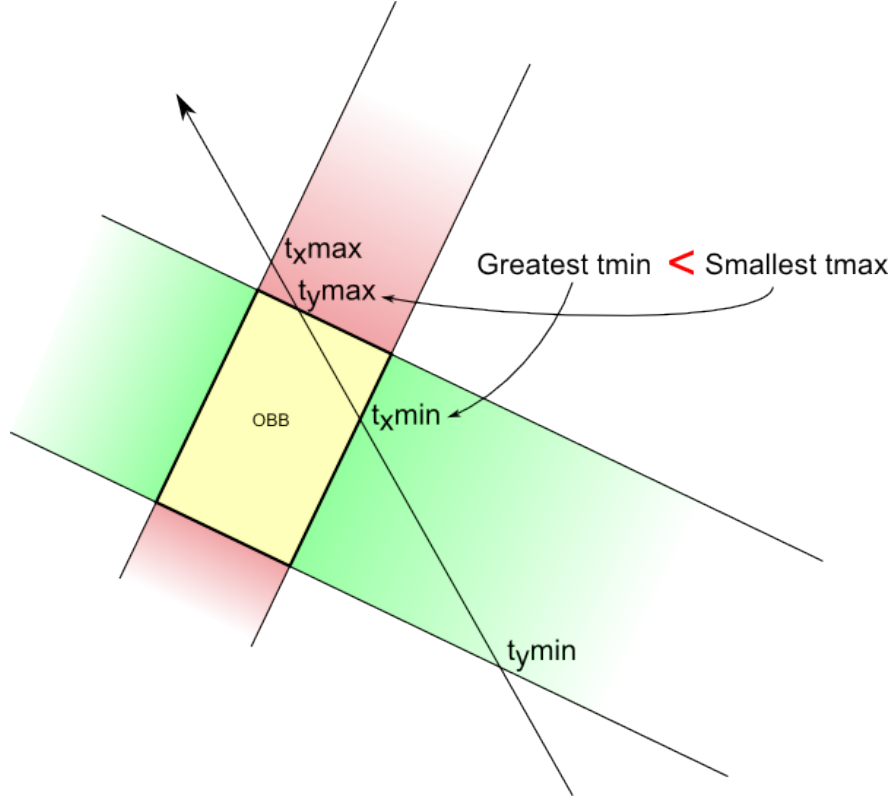


Figure 12: Ray hits the bounding box.

If an intersection is spotted, we calculate the ray's intersection point using the following formula:

Let **rayOrigin** and **rayDirection** hold a ray's Starting point and Direction respectively.

$$IntersectionPoint = \frac{rayOrigin + (rayDirection * Distance)}{\sqrt{rayDirection.x^2 + rayDirection.y^2 + rayDirection.z^2}} \quad (1)$$

Finally, each intersection point is stored temporarily whenever there is an intersection. Before doing so, we subtract the current with the previous intersection value and use it to apply transformations on the selected entity.

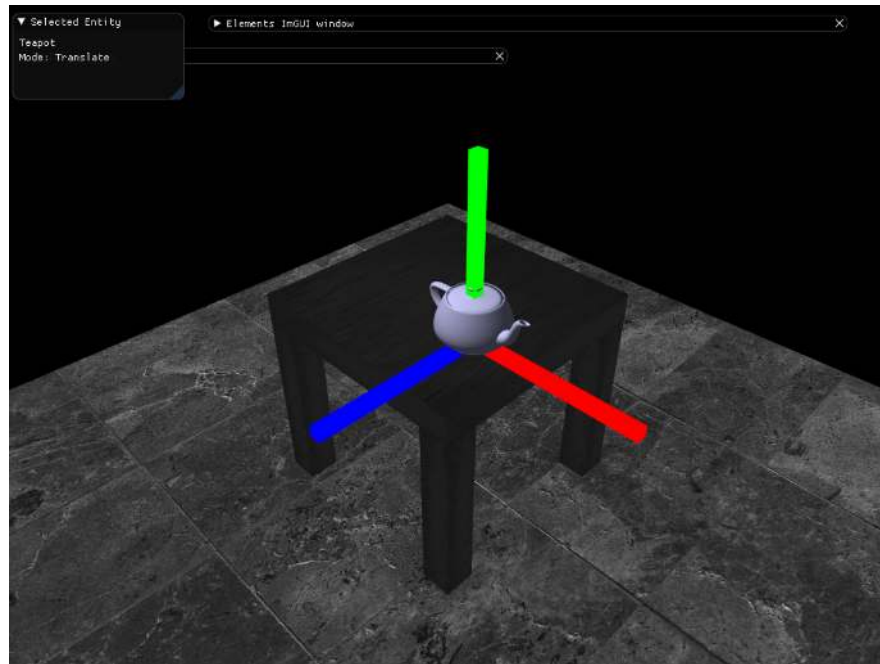


Figure 13: Translation gizmos in Elements

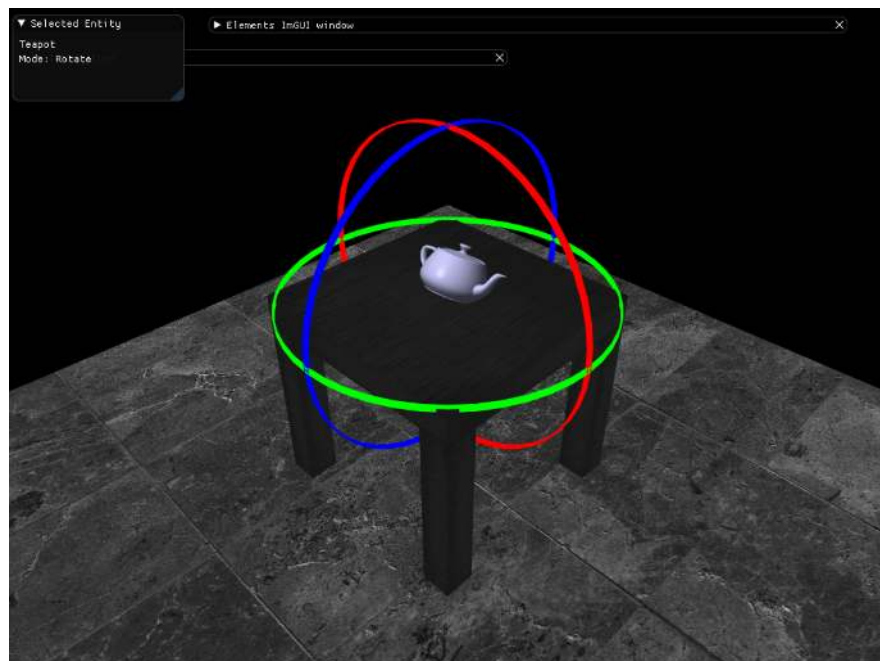


Figure 14: Rotation gizmos in Elements

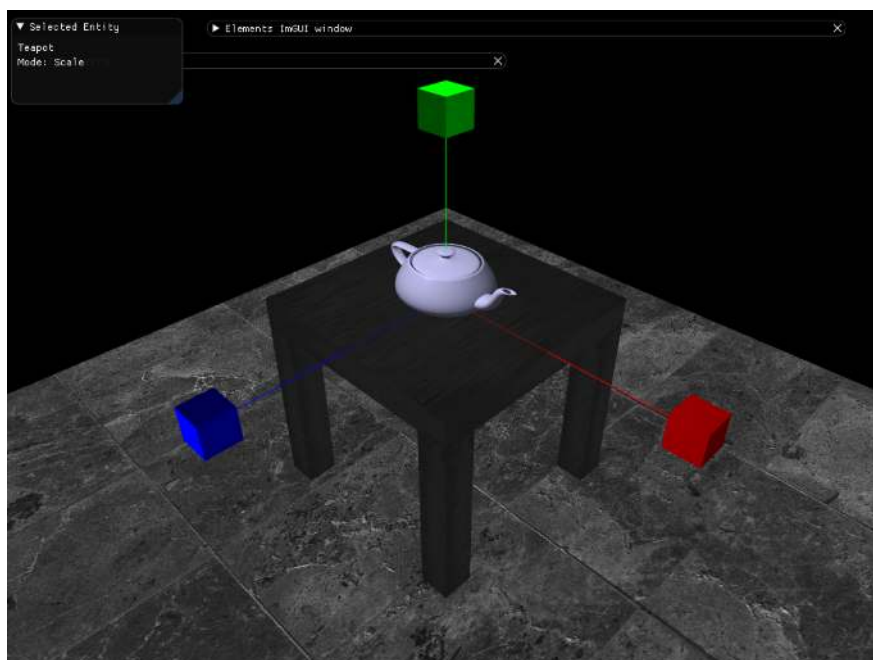


Figure 15: Scaling gizmos in Elements

4 ElementsXR

4.1 Intro

The goal of this thesis as implied before, was to utilize the gizmos functionality in virtual reality. This time rather than using the mouse cursor as a starting point, we used the controllers' initial position instead.

4.2 Our Approach

Before anything else, we had to connect the Elements scene with the OpenXR runtime. To accomplish that, we used the OpenXR's wrapper for python, `py-openxr`, which provides various methods similar to those used in `c++`. These methods allow us to create programs with various configurations that can be recognized by the OpenXR runtime. The next and final stage of this project was to implement the Gizmos utilities inside virtual reality.

4.3 Technical details

4.3.1 Connecting the scene to the OpenXR runtime

In order to render the Elements scene inside an OpenXR runtime our program should include the following components:

- a platform plugin, which is used to acquire the platform extensions based on the platform the program is operating.
- a graphics plugin, which is used for entity initialization and rendering.
- an options component, which holds information about several configurations (See Appendix A.3).

Since this project was intended for use in virtual reality, we used the following configuration:

- Environment Blend Mode: Opaque
- Form Factor: Head Mounted Display
- View Configuration: Stereo

Before starting the rendering loop, an instance of the program must be created, that is recognized by the OpenXR runtime. More specifically we must:

- Create an OpenXR Instance so that the OpenXR runtime can use this app.

- Select a system for the given view configuration
- Initialize the scene's Entities.
- Initialize an OpenXR session and actions which are used to acquire information about the controllers.
- Create a swapchain for each view. swapchains include images provided by the run-time, for an application to render into.

After the initialization is completed rendering can begin. The scene that was used for this demo includes a skybox, a floor and a table on which there is a teapot (See Fig. 16). When the controllers are activated, we can also see 2 cubes representing the user's hands.

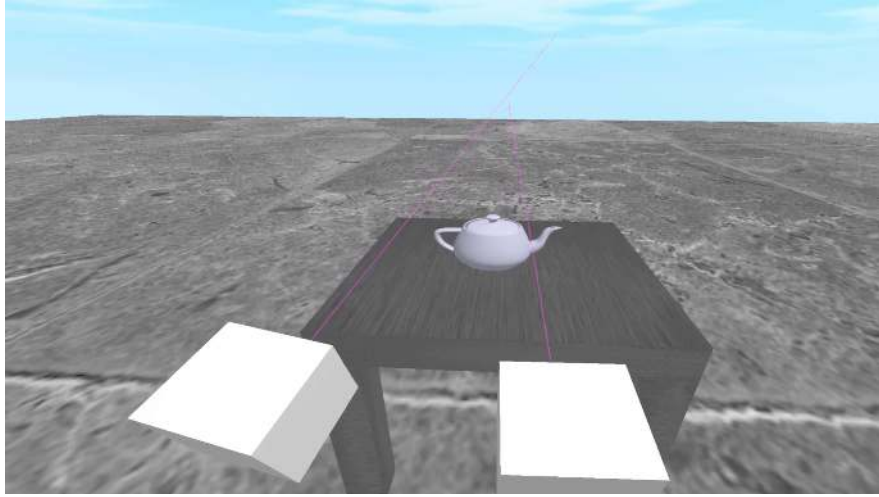


Figure 16: Elements scene inside the OpenXR runtime.

4.3.2 Gizmos in ElementsXR

Virtual reality gizmos use the same principles as those on screen. The only difference is that the ray-casting happens directly from the world space. Therefore, we do not have to change coordinates spaces like in normal Gizmos.

In our implementation, we use the right controller to cast rays and the left controller to modify entities and/or transformations. To determine the ray's direction, we first need to find the ray's starting and ending points in world space coordinates by using the ray's mesh and transformation matrix, and then subtract the latter from the former.

$$rayCoordinates = rayTransformMatrix * rayMesh \quad (2)$$

The result above gives us the ray's world space coordinates used for ray-casting. Finally, this data is used to evaluate whether the rays starting from the user's hands hit the gizmos' bounding boxes in order to apply the appropriate transformations (See Fig. 17, 18 and 19).

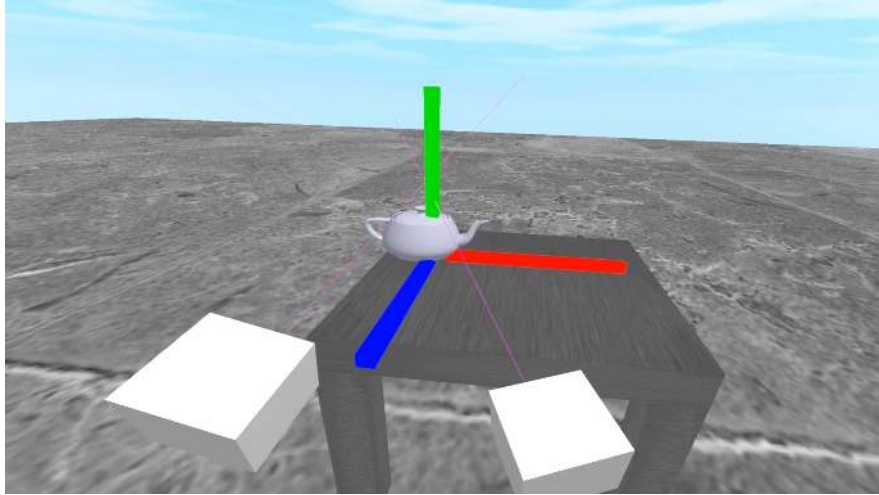


Figure 17: Translation inside OpenXR.

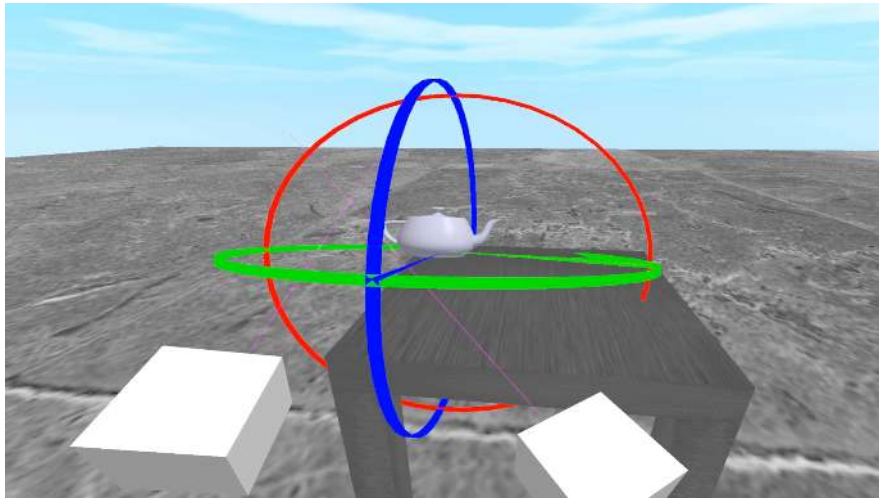


Figure 18: Rotation inside OpenXR.

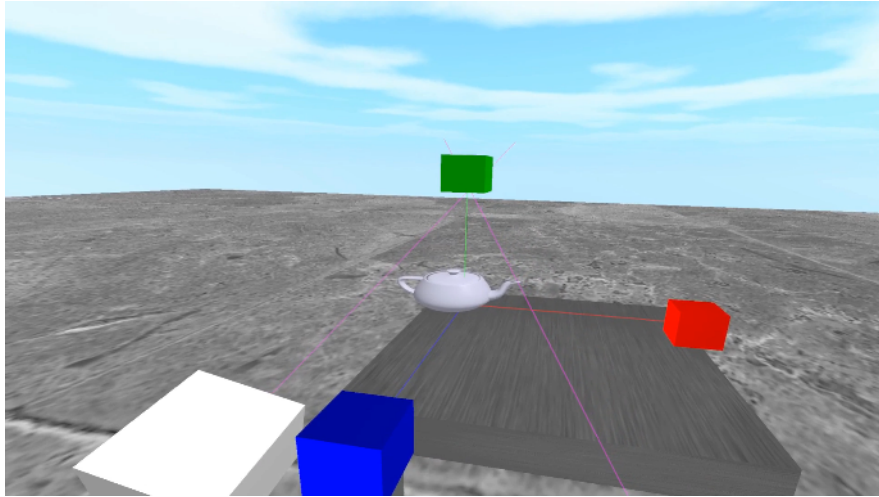


Figure 19: Scaling inside OpenXR.

5 Conclusion

5.1 Summary

This project's goal was to develop gizmos that can be used for scene manipulation inside virtual reality. To achieve that, we used ray-casting and intersection algorithms, that allowed us to apply transformations on entities on a scene. The scene that was rendered includes a skybox, a floor and a table with a teapot on it.

5.2 Limitations and future work

The current demo allows the user to change selected entities by pressing one of the controller's buttons, but they can not select different entities using ray-casting. Therefore, while they are able to apply transformations to different objects, they can not select them themselves. So an interesting feature would be to implement object picking inside Virtual Reality, in the interest of giving the user more freedom when interacting with the scene while wearing a headset.

References

- 2d textures reference.* (n.d.). <https://learnopengl.com/Getting-started/Textures>.
- 3d textures reference.* (n.d.). <https://learnopengl.com/Advanced-OpenGL/Cubemaps>.
- Computer graphics.* (n.d.). https://en.wikipedia.org/wiki/Computer_graphics.
- Data visualization photo source.* (n.d.). <https://builtin.com/data-science/data-visualization>.
- Graphics tablet photo source.* (n.d.). <https://www.xp-pen.com/blog/digital-art-programs-for-drawing-painting-illustration.html>.
- Official openxr example.* (n.d.). <https://github.com/KhronosGroup/OpenXR-SDK-Source>.
- Papagiannakis, G., Kamarianakis, M., Protopsaltis, A., Angelis, D., & Zikas, P. (2023). Project Elements: A Computational Entity-component-system in a Scene-graph Pythonic Framework, for a Neural, Geometric Computer Graphics Curriculum. In A. Magana & J. Zara (Eds.), *Eurographics 2023 - education papers*. The Eurographics Association. doi: 10.2312/eged.20231015
- pyopenxr examples repository.* (n.d.). https://github.com/cmbruns/pyopenxr_examples.
- Ray-bounding box intersection.* (n.d.). <http://www.opengl-tutorial.org/miscellaneous/clicking-on-objects/picking-with-custom-ray-obb-function/>.
- Unity editor.* (n.d.). <https://unity.com/roadmap/unity-platform>.
- Unreal engine 5 editor.* (n.d.). <https://docs.unrealengine.com/5.2/en-US/level-editor-in-unreal-engine/>.

A Appendix

A.1 OpenXR Instructions OpenXR Instructions for dummies

SteamVR (Skip bullets if you use a Head Mounted Display)
Download steamVR via steam

1. Run SteamVR once (Running SteamVR for the first time will create a file that you can edit)
2. Go to this file : `ProgramFiles(x86)\Steam\config\steamvr.vrsettings` and add the following lines to “steamvr.vrsettings”. In the steamvr section edit the following values:
`"forcedDriver" : "null",`
`"activateMultipleDrivers" : "true"`
3. Go to this file : `ProgramFiles(x86)\Steam\steamapps\common\SteamVR\drivers\null\resources\settings\default.vrsettings` And change the “enable” value from “false” to “true” Additionally remove the following lines (otherwise your program might crash)
`"windowX": 0,`
`"windowY": 0,`
`"windowWidth": 2160,`
`"windowHeight": 1200,`

OpenSDK repository on github

1. Download or fork and clone the following repository
<https://github.com/KhronosGroup/OpenXR-SDK-Source>
2. Open the repository in visual studio 2019 (2013,2015,2017 and 2022 versions work too, for more information about these versions check out BUILDING.md in the repository)
3. Open a Terminal in Visual Studio (I used the Developer powershell) and execute the following commands:
`mkdir build/win64`
`cd build/win64`
4. execute the following command
`cmake -G "Visual studio 16 2019" ..:`
This will build the OpenXR API with the examples
5. Open build/win64 in visual studio 2019, right click and build “ALL_BUILD.vcxproj”. This will build the openxr loader and the examples.

Run the hello_xr executable

1. Run steamVR
2. Go to the steamVR settings
3. Go to video settings and turn off the “Pause VR when headset is idle” choice
4. open a command prompt (I used windows Powershell)
5. Go to this path : `OpenXR-SDK-Source-main\build\win64\src\tests\hello_xr\Debug` and execute the following command (with steamVR running)
`./hello_xr -g OpenGL`
6. If you select Display VR View you should be able to see the result (see Fig. 20).

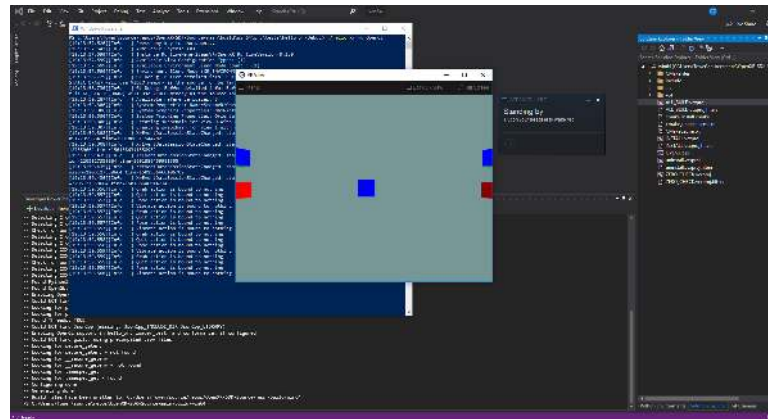


Figure 20: OpenXR Example

A.2 Gizmos

Gizmos Sample code

```
def change_target(self):
    """
    Change selected entity
    Arguments:
        self: self
    Returns:
        None
    """
    self.selected = self.selected+1
    count = self.selected
    if(count>self.total):
        count = 1
        self.selected = 1

    for component in self.scene.world.root:

        #Have to check because there is always some component that is NoneType
        if component is not None:
            parentname = component.parent.name
            #next BasicTransform component that is not one of the gizmos components and is not child of
            #the camera in use
            if component.getClassName()=="BasicTransform" and component.name not in self.gizmos_comps
            and parentname!=self.cameraInUse and parentname.find("Skybox")==-1:
                count = count-1
                if (count==0):
                    self.selected_trans = component
                    self.selected_comp = self.selected_trans.parent.name
                    children = component.parent.getNumberOfChildren()
                    self.selected_mesh = None
                    for i in range(children):
                        child = component.parent.getChild(i)
                        if child is not None and child.getClassName()=="RenderMesh":
                            self.selected_mesh = child
                            break
                    break

def update_projection(self, Proj):
    """
    Update window's projection and calculate its inverse, if needed
    Arguments:
        self: self
        Proj: Projection matrix
    Returns:
        None
    """
    if self.selected is not None and not np.array_equiv(self.projection, Proj):
        self.projection = Proj
        self.inv_projection = util.inverse(self.projection)
        self.__update_gizmos()

def update_view(self, View):
    """
    Update window's View and calculate its inverse, if needed
    Arguments:
        self: self
        View: View matrix
    Returns:
        None
    """
    if self.selected is not None and not np.array_equiv(self.view, View):
        self.view = View
        self.inv_view = util.inverse(self.view)
        self.__update_gizmos()

def calculate_bounding_box(self, mesh_vertices):
    """
    A simple method that calculates an axis aligned bounding box using a given mesh's vertices
    Arguments:
        self: self
        mesh: A RenderMesh component
    Returns
        minbb: minimum bounding box coordinates
        maxbb: maximum bounding box coordinates
    """
    vertices = mesh_vertices
```

```

for i in range(len(vertices)):
    vertices[i] = vertices[i]/vertices[i][3]

minbb = util.vec(vertices[0][0], vertices[0][0], vertices[0][2], 1.0)
maxbb = util.vec(vertices[0][0], vertices[0][0], vertices[0][2], 1.0)
for i in range(1, len(vertices)):
    #min coordinates
    if vertices[i][0] < minbb[0]:
        minbb[0] = vertices[i][0]
    if vertices[i][1] < minbb[1]:
        minbb[1] = vertices[i][1]
    if vertices[i][2] < minbb[2]:
        minbb[2] = vertices[i][2]

    #max coordinates
    if vertices[i][0] > maxbb[0]:
        maxbb[0] = vertices[i][0]
    if vertices[i][1] > maxbb[1]:
        maxbb[1] = vertices[i][1]
    if vertices[i][2] > maxbb[2]:
        maxbb[2] = vertices[i][2]
return minbb, maxbb

def calculate_ray(self):
    """
    Calculate and Return a Ray that starts from mouse position
    Arguments:
        self: self
    Returns:
        a ray's starting position and direction

    Source: http://www.opengl-tutorial.org/miscellaneous/clicking-on-objects/
    picking-with-custom-ray-obb-function/
    """

    #mouse position in normalized device coordinates
    x = 2.0 * (self.mouse_x.value/self.screen_width - 0.5)
    y = -2.0 * (self.mouse_y.value/self.screen_height - 0.5)

    #ray start and ray end in normalized device coordinates
    ray_start = util.vec(x, y, -1.0, 1.0)
    ray_end = util.vec(x, y, 0.0, 1.0)

    # normalized device to Camera space
    ray_start_Camera = self.inv_projection @ ray_start
    ray_start_Camera = ray_start_Camera/ray_start_Camera[3]
    # Camera space to world space
    ray_start_World = self.inv_view @ ray_start_Camera
    ray_start_World = ray_start_World/ray_start_World[3]

    #normalized device to Camera space
    ray_end_Camera = self.inv_projection @ ray_end
    ray_end_Camera = ray_end_Camera/ray_end_Camera[3]

    # Camera space to World space
    ray_end_world = self.inv_view @ ray_end_Camera
    ray_end_world = ray_end_world/ray_end_world[3]

    #calculate and normalize the ray's direction
    ray_dir_world = util.vec(ray_end_world[0] - ray_start_World[0],
                             ray_end_world[1] - ray_start_World[1],
                             ray_end_world[2] - ray_start_World[2])
    ray_dir_world = util.normalise(ray_dir_world)
    ray_origin = util.vec(ray_start_World[0], ray_start_World[1], ray_start_World[2], 0.0)
    ray_direction = util.vec(ray_dir_world[0], ray_dir_world[1], ray_dir_world[2], 0.0)

    return ray_origin, ray_direction

```

A.3 OpenXR Options

View Configuration	
Name	Description
Mono	One view representing the form factor's one primary display
Stereo	Two views representing the form factor's two primary displays, which map to a left-eye and right-eye view

Form Factor	
Name	Description
Head Mounted Display	The tracked display is attached to the user's head
Handheld Display	The tracked display is held in the user's hand, independent from the user's head

Environment Blend Mode	
Name	Description
Opaque	The composition layers will be displayed with no view of the physical world behind them
Additive	The composition layers will be additively blended with the real world behind the display
Alpha Blend	The composition layers will be alpha-blended with the real world behind the display

A.4 ElementsXR Sample code

```
def Initialize(self, name: str, renderer : InitGLShaderSystem):
    """
    All Initializations packed inside a single method
    Arguments:
        self: self
        name: Application name
        renderer: System that initializes Entities on the scene
    Returns:
        None
    """
    self.createInstance(name)
    self.InitializeSystem()
    self.InitializeDevice(renderer)
    self.InitializeSession()
    self.create_Swapchains()

def createInstance(self, name: str):
    """
    create instance of the OpenXR program
    Arguments:
        self: self
        name: Application name
    Returns:
        None
    """
    self.log_layers_and_extensions()

    assert self.instance is None

    # Create union of extensions required by platform and graphics plugins.
    extensions = []
    # Enable debug messaging
    discovered_extensions = xr.enumerate_instance_extension_properties()
    dumci = xr.DebugUtilsMessengerCreateInfoEXT()
    next_structure = self.platform_plugin.instance_create_extension
    ALL_SEVERITIES = (
        xr.DEBUG_UTILS_MESSAGE_SEVERITY_VERBOSE_BIT_EXT
        | xr.DEBUG_UTILS_MESSAGE_SEVERITY_INFO_BIT_EXT
        | xr.DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT
        | xr.DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT
    )
```

```

ALL_TYPES = (
    xr.DEBUG_UTILS_MESSAGE_TYPE_GENERAL_BIT_EXT
    | xr.DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT
    | xr.DEBUG_UTILS_MESSAGE_TYPE_PERFORMANCE_BIT_EXT
    | xr.DEBUG_UTILS_MESSAGE_TYPE_CONFORMANCE_BIT_EXT
)

if xr.EXT_DEBUG_UTILS_EXTENSION_NAME in discovered_extensions:
    extensions.append(xr.EXT_DEBUG_UTILS_EXTENSION_NAME)
    dumci.message_severities = ALL_SEVERITIES
    dumci.message_types = ALL_TYPES
    dumci.user_data = None
    dumci.user_callback = self.debug_callback
    if next_structure is None:
        next_structure = cast(pointer(dumci), c_void_p)
    else:
        next_structure.next = cast(pointer(dumci), c_void_p)

#
extensions.extend(self.platform_plugin.instance_extensions)
extensions.extend(self.graphics_plugin.instance_extensions)

self.instance = xr.InstanceObject(
    enabled_extensions=extensions,
    application_name=name,
    application_version=xr.Version(0, 0, 1),
    next=next_structure,
)
self.log_instance_info()

def InitializeSystem(self):
    """
    Select a System for the view configuration specified in the Options and initialize the graphics
    device for the selected
    system.
    Arguments:
        self: self
    Returns:
        None
    """
    assert self.instance is not None
    assert self.system is None
    form_factor = options.get_xr_form_factor(self.options.form_factor)
    self.system = xr.SystemObject(instance=self.instance, form_factor=form_factor)
    print(f"Using system {hex(self.system.id.value)} for form factor {str(form_factor)}")
    assert self.instance.handle is not None
    assert self.system.id is not None

def InitializeDevice(self,
                    renderer : InitGLShaderSystem):
    self.log_view_configurations()
    """
    The graphics Plugin uses the initialization system to initialize the required objects for the scene
    Arguments:
        self: self
        renderer: System that initializes Entities on the scene
    Returns:
        None
    """
    self.graphics_plugin.initialize_device(self.instance.handle, self.system.id, renderer)

def InitializeSession(self):
    """
    Create a Session and other basic session-level initialization.
    Arguments:
        self: self
    Returns:
        None
    """
    assert self.instance is not None
    assert self.instance.handle != xr.NULL_HANDLE
    assert self.session is None

    print(f"Creating session...")

    graphics_binding_pointer = cast(
        pointer(self.graphics_plugin.graphics_binding),
        c_void_p)
    create_info = xr.SessionCreateInfo(
        next=graphics_binding_pointer,
        system_id=self.system.id,

```

```

)
self.session = xr.create_session(
    instance=self.instance.handle,
    create_info=create_info,
)
self.log_reference_spaces()

self.initialize_actions()

self.create_visualized_spaces()
self.app_space = xr.create_reference_space(
    session=self.session,
    create_info=get_xr_reference_space_create_info(self.options.app_space),
)

def create_Swapchains(self):
    """
    Create a Swapchain for each view which require coordinating with the graphics plugin to select
    the format, getting the system graphics properties, getting the view configuration and
    grabbing the resulting swapchain images.
    Arguments:
        self: self
    Returns:
        None
    """

    assert self.session is not None
    assert len(self.swapchains) == 0
    assert len(self.config_views) == 0

    system_properties = xr.get_system_properties(self.instance.handle, self.system.id)

    print("System Properties: "
          f"Name={system_properties.system_name.decode()} "
          f"VendorId={system_properties.vendor_id}")
    print("System Graphics Properties: "
          f"MaxWidth={system_properties.graphics_properties.max_swapchain_image_width} "
          f"MaxHeight={system_properties.graphics_properties.max_swapchain_image_height} "
          f"MaxLayers={system_properties.graphics_properties.max_layer_count}")
    print("System Tracking Properties: "
          f"OrientationTracking={bool(system_properties.tracking_properties.orientation_tracking)} "
          f"PositionTracking={bool(system_properties.tracking_properties.position_tracking)}")

    self.config_views = xr.enumerate_view_configuration_views(
        instance=self.instance.handle,
        system_id=self.system.id,
        view_configuration_type=self.options.parsed["view_config_type"],
    )
    # Create and cache view buffer for xrLocateViews later.
    view_count = len(self.config_views)
    assert view_count == 2 # 2 eyes
    self.views = (xr.View * view_count)(*[xr.View()] * view_count)

    # Create the swapchain and get the images.
    # Select a swapchain format.
    swapchain_formats = xr.enumerate_swapchain_formats(self.session)
    self.color_swapchain_format = self.graphics_plugin.select_color_swapchain_format(swapchain_formats)

    formats_string = ""
    for sc_format in swapchain_formats:
        selected = sc_format == self.color_swapchain_format
        formats_string += " "
        if selected:
            formats_string += "["
            formats_string += f"{str(self.color_swapchain_format)}({sc_format})"
            formats_string += "]"
        else:
            formats_string += str(sc_format)
    print(f"Swapchain Formats: {formats_string}")

    # Create a swapchain for each view.
    for i, vp in enumerate(self.config_views):
        print("Creating swapchain for "
              f"view {i} with dimensions "
              f"Width={vp.recommended_image_rect_width} "
              f"Height={vp.recommended_image_rect_height} "
              f"SampleCount={vp.recommended_swapchain_sample_count}")

        # Create the swapchain.
        swapchain_create_info = xr.SwapchainCreateInfo(

```

```

        array_size=1,
        format=self.color_swapchain_format,
        width=vp.recommended_image_rect_width,
        height=vp.recommended_image_rect_height,
        mip_count=1,
        face_count=1,
        sample_count=self.graphics_plugin.get_supported_swapchain_sample_count(vp),
        usage_flags=xr.SwapchainUsageFlags.SAMPLED_BIT | xr.SwapchainUsageFlags.COLOR_ATTACHMENT_BIT,
    )

    swapchain = Swapchain(
        xr.create_swapchain(
            session=self.session,
            create_info=swapchain_create_info,
        ),
        swapchain_create_info.width,
        swapchain_create_info.height,
    )

    self.swapchains.append(swapchain)
    swapchain_image_buffer = xr.enumerate_swapchain_images(
        swapchain=swapchain.handle,
        element_type=self.graphics_plugin.swapchain_image_type,
    )

    # Keep the buffer alive by moving it into the list of buffers.
    self.swapchain_image_buffers.append(swapchain_image_buffer)
    capacity = len(swapchain_image_buffer)
    swapchain_image_ptr_buffer = (POINTER(xr.SwapchainImageBaseHeader) * capacity)()
    for ix in range(capacity):
        swapchain_image_ptr_buffer[ix] = cast(
            byref(swapchain_image_buffer[ix]),
            POINTER(xr.SwapchainImageBaseHeader)
        )
    self.swapchain_image_ptr_buffers[hash(swapchain.handle)] = swapchain_image_ptr_buffer

def render_frame(self, renderer: RenderGLShaderSystem) -> None:
    """
    Create and submit a frame.
    Arguments:
        self: self
        renderer: Rendering System that the graphics plugin uses to update uniform variables
    Returns:
        None
    """
    assert self.session is not None
    frame_state = xr.wait_frame(
        session=self.session,
        frame_wait_info=xr.FrameWaitInfo(),
    )
    xr.begin_frame(self.session, xr.FrameBeginInfo())

    layers = []
    layer_flags = 0
    if self.options.environment_blend_mode == "AlphaBlend":
        layer_flags = xr.COMPOSITION_LAYER_BLEND_TEXTURE_SOURCE_ALPHA_BIT \
            | xr.COMPOSITION_LAYER_UNPREMULTIPLIED_ALPHA_BIT
    layer = xr.CompositionLayerProjection(
        space=self.app_space,
        layer_flags=layer_flags,
    )
    projection_layer_views = (xr.CompositionLayerProjectionView * 2)(
        xr.CompositionLayerProjectionView(),
        xr.CompositionLayerProjectionView()
    )
    if frame_state.should_render:
        if self.render_layer(frame_state.predicted_display_time, projection_layer_views, layer, renderer):
            layers.append(byref(layer))

    xr.end_frame(
        session=self.session,
        frame_end_info=xr.FrameEndInfo(
            display_time=frame_state.predicted_display_time,
            environment_blend_mode=self.options.parsed["environment_blend_mode"],
            layer_count=len(layers),
            layers=layers,
        ),
    )

def render_layer(self,
    predicted_display_time: xr.Time,

```

```

        projection_layer_views: Array,
        layer: xr.CompositionLayerProjection,
        renderer: RenderGLShaderSystem
    ) -> bool:
"""
Render and return a layer
Arguments:
    self: self
    predicted_display_time:
    projection_layer_views: array containing information about each view
    layer: layer to be rendered
    renderer: Rendering System that the graphics plugin uses to update uniform variables
Returns:
    True if the layer could be rendered, False otherwise
"""
view_capacity_input = len(self.views)
view_state, self.views = xr.locate_views(
    session=self.session,
    view_locate_info=xr.ViewLocateInfo(
        view_configuration_type=self.options.parsed["view_config_type"],
        display_time=predicted_display_time,
        space=self.app_space,
    ),
)
view_count_output = len(self.views)
vsf = view_state.view_state_flags
if (vsf & xr.VIEW_STATE_POSITION_VALID_BIT == 0
    or vsf & xr.VIEW_STATE_ORIENTATION_VALID_BIT == 0):
    return False # No valid tracking poses for the views.
assert view_count_output == view_capacity_input
assert view_count_output == len(self.config_views)
assert view_count_output == len(self.swapchains)
assert view_count_output == len(projection_layer_views)

model_head = util.identity()

if self.head is not None:
    model_head = self.head.getChild(0).l2world

#Update hands before rendering
for hand in Side:
    space_location = xr.locate_space(
        space=self.input.hand_space[hand],
        base_space=self.app_space,
        time=predicted_display_time,
    )
    loc_flags = space_location.location_flags

    #if true then the program has focus -> it can take input from the controllers
    if (loc_flags & xr.SPACE_LOCATION_POSITION_VALID_BIT != 0
        and loc_flags & xr.SPACE_LOCATION_ORIENTATION_VALID_BIT != 0):
        scale = 0.1 * self.input.hand_scale[hand]
        #scale = 1.0
        position = space_location.pose.position
        orientation = space_location.pose.orientation

        between_hands = self.hand_dist
        if not hand: #Left hand goes a bit to the left, while the right hand goes a bit to the right.
            between_hands = -between_hands
        model = model_head
        @ util.translate(position.x-0.2+between_hands, position.y+4.0, position.z+6.3)
        @ util.inverse(create_xr_quaternion(util.quaternion(orientation.x,
                                                                orientation.y,
                                                                orientation.z,
                                                                orientation.w)))

        @ util.scale(scale, scale, scale)
        self.hands[hand].getChildByType(BasicTransform.getClassName()).trs = model

# Render view to the appropriate part of the swapchain image.
for i in range(view_count_output):
    view_swapchain = self.swapchains[i]
    swapchain_image_index = xr.acquire_swapchain_image(
        swapchain=view_swapchain.handle,
        acquire_info=xr.SwapchainImageAcquireInfo(),
    )
    xr.wait_swapchain_image(
        swapchain=view_swapchain.handle,
        wait_info=xr.SwapchainImageWaitInfo(timeout=xr.INFINITE_DURATION),
    )
    view: xr.CompositionLayerProjectionView

```

```

view = projection_layer_views[i]

assert view.type == xr.StructureType.COMPOSITION_LAYER_PROJECTION_VIEW
view.pose = self.views[i].pose
view.fov = self.views[i].fov
view.next = None
view.sub_image.image_array_index = 0
view.sub_image.swapchain = view_swapchain.handle
view.sub_image.image_rect.offset[:] = [0, 0]
view.sub_image.image_rect.extent[:] = [view_swapchain.width, view_swapchain.height]
swapchain_image_ptr = self.swapchain_image_ptr_buffers[
hash(view_swapchain.handle)][swapchain_image_index]

offset = util.vec(0.064, 0.0, 0.0)
if i==1:
    offset = -offset

self.graphics_plugin.Render_View(
    layer_view=view,
    swapchain_image_base_ptr=swapchain_image_ptr,
    _swapchain_format=self.color_swapchain_format,
    renderUpdate=renderer,
    offset=offset,
    mirror=False #mirror=i==0 #mirror left eye only
)
xr.release_swapchain_image(
    swapchain=view_swapchain.handle,
    release_info=xr.SwapchainImageReleaseInfo()
)
layer.view_count = len(projection_layer_views)
layer.views = projection_layer_views
return True

```